

Human Cell Digital Twin

Core process descriptions, governing equations, and implementation blocks

You built a minimum viable biological simulation for a human-cell digital twin. The current code is a compartmental ODE model: one state dictionary holds molecule concentrations, each process computes a flux for one time step, and the main loop applies those fluxes in biological order. The model is designed to become spatial by running the same reaction equations independently in each voxel and adding a diffusion Laplacian.

What Exists So Far

Layer	Files	Role
Parameters	constants.py	Central kinetic constants, time step, duration, and process rates.
State	state.py	Initial mM concentrations for ATP, ADP, glucose, oxygen, ions, calcium, amino acids, protein mass, and HIF-1a.
Math helpers	helpers.py	Clamp, Michaelis-Menten saturation, and Hill functions.
Processes	p01...p09	Nine biological flux modules with compute/apply functions.
Runner	simulate.py	Scenario controller and process ordering for normal, hypoxia, and starvation runs.
QA/viz	validate.py, plot.py	Sanity checks and time-series plots for the hypoxia demo.

Current Implementation Note

The process filenames have been normalized to Python-importable module names, matching simulate.py. The simulation runs, and the current validation status is 10/10 checks passing after making GLUT reversible and ADP-limiting ATP-producing reactions.

Six-Week Execution Plan

The plan keeps the deliverable intentionally narrow: nine process ODEs, HIF-1a hypoxia coupling, a 3-D reaction-diffusion grid, GPU scaling, WebGL volume rendering, and a hypoxia demo. Calcium, apoptosis, transcription/translation, Poisson electrostatics, waste/autophagy, and cytoplasmic flow are explicitly deferred.

6-week plan — 4 parallel tracks

	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6
Biology lead	All 9 processes ODE model in Python		HIF-1 α coupling	Validate hypoxia test	Support viz + demo Biology commentary	
GPU / sim	Env setup CuPy, CUDA	Laplacian stencil 3D kernel, ping-pong		Stability sub-stepping	WGSL compute shader Feed buffer to viz	
Voxel pair	1D proto O ₂ diffusion	3D grid 32 ³ NumPy	Graft ODEs R(C) per voxel	Membrane level-set bdry	Scale to 100 ³ Hand off to GPU lead	
Viz lead	three.js scaffold Volume render 32 ³ array		Color maps ATP, O ₂ , Na ⁺	Inject UI Click → drop O ₂	Unity HoloLens port	Demo polish Open house
Milestones	End wk 1 ODE runs, ATP stable at rest		End wk 3 Spatial 32 ³ runs + viz shows wave		End wk 5 100 ³ on GPU, WebGL live, Unity started	End wk 6 Open house hypoxia demo live
Cut from scope — not in 6 weeks from zero Calcium signaling · Apoptosis / cell fate · Stokes flow (cytoplasmic streaming) Poisson electrostatics · Waste / autophagy · Transcription / translation HoloLens is stretch goal only — WebGL is the real deliverable						
Keep — minimum viable demo Glycolysis · OxPhos · Fermentation · ATP budget · mTOR (Proc 8) · Na/K pump (Proc 9) HIF-1 α O ₂ coupling · 3D reaction-diffusion grid · WebGL volume render · hypoxia scenario						

Core Processes

Process 1: Membrane Transport

Imports glucose and oxygen, and models passive sodium leak across the membrane. HIF-1a scales GLUT activity under hypoxia.

Governing equations:

$$-v_{\text{glut}} = V_{\text{max_glut}} * \text{glut_scale} * \text{Glc_ext} / (\text{Km_glut} + \text{Glc_ext})$$

$$-v_{\text{o2}} = V_{\text{max_o2}} * \text{O2_ext} / (\text{Km_o2} + \text{O2_ext})$$

$$-\text{glut_scale} = 1 + (\text{HIF_MAX_GLUT} - 1) * \text{hif1a}$$

Implementation excerpt:

```
def compute(s, dt, params):
    """
    Parameters
    -----
    s      : state dict
    dt     : time step (minutes)
    params : scenario dict with keys 'glucose_ext', 'O2_ext'

    Returns
    -----
    fluxes : dict of (flux_name -> value) – NOT yet applied to state.
    """
    hif = s["hif1a"]
    glut_scale = 1.0 + (HIF_MAX_GLUT - 1.0) * hif

    ext_drive = mm(params["glucose_ext"], KM_GLUT_GLU)
    cyt_drive = mm(s["glucose"], KM_GLUT_GLU)
    v_glut = VMAX_GLUT * glut_scale * (ext_drive - cyt_drive) * dt
    # GLUT is reversible: positive flux imports, negative flux exports.
    v_glut = min(v_glut, max(params["glucose_ext"], 0.0))
    v_glut = max(v_glut, -s["glucose"])

    v_o2 = VMAX_O2_IMPORT * mm(params["O2_ext"], KM_O2_EXT) * dt

    v_na = NA_LEAK_RATE * dt

    return {
        "v_glut": v_glut,
        "v_o2": v_o2,
        "v_na_leak": v_na,
    }

def apply(s, fluxes):
    """Commit transport fluxes to state."""
    s["glucose"] += fluxes["v_glut"]
    s["O2"] += fluxes["v_o2"]
    s["Na_cyt"] += fluxes["v_na_leak"]
    s["Na_ext"] -= fluxes["v_na_leak"]
    return s
```

Process 2: Glycolysis

Converts cytosolic glucose into pyruvate and a small amount of ATP. ADP activates the flux, approximating PFK demand coupling.

Governing equations:

$$-v_{\text{glyc}} = V_{\text{max_glyc}} \cdot \text{Glc} / (K_{\text{m_glc}} + \text{Glc}) \cdot \text{ADP} / (K_{\text{m_adp}} + \text{ADP})$$



Implementation excerpt:

```
def compute(s, dt):
    v_glyc = VMAX_GLYC * mm(s["glucose"], KM_GLYC_GLU) * mm(s["ADP"], KM_GLYC_ADP) * dt
    v_glyc = min(v_glyc, s["glucose"]) # can't consume more glucose than available
    v_glyc = min(v_glyc, s["ADP"] / ATP_PER_GLYC) # ATP production is ADP-limited
    return {"v_glyc": v_glyc}

def apply(s, fluxes):
    v = fluxes["v_glyc"]
    s["glucose"] -= v
    s["pyruvate"] += PYR_PER_GLYC * v
    s["ATP"] += ATP_PER_GLYC * v
    s["ADP"] -= ATP_PER_GLYC * v
    return s
```

Process 3: Oxidative Phosphorylation

Uses pyruvate and oxygen to produce the main ATP supply under normoxia.

Governing equations:

$$-v_{\text{oxphos}} = V_{\text{max_oxphos}} \cdot \text{Pyr} / (K_{\text{m_pyr}} + \text{Pyr}) \cdot \text{O}_2 / (K_{\text{m_o}_2} + \text{O}_2)$$



Implementation excerpt:

```
def compute(s, dt):
    v = VMAX_OXPHOS * mm(s["pyruvate"], KM_OXPHOS_PYR) * mm(s["O2"], KM_OXPHOS_O2) * dt
    # Substrate clamps – cannot consume more than available
    v = min(v, s["pyruvate"])
    v = min(v, s["O2"] / max(O2_PER_OXPHOS, 1e-9))
    v = min(v, s["ADP"] / max(ATP_PER_OXPHOS, 1e-9))
    return {"v_oxphos": v}

def apply(s, fluxes):
    v = fluxes["v_oxphos"]
    s["pyruvate"] -= v
    s["O2"] -= O2_PER_OXPHOS * v
    s["ATP"] += ATP_PER_OXPHOS * v
    s["ADP"] -= ATP_PER_OXPHOS * v
    return s
```

Process 4: Fermentation

Converts pyruvate to lactate when oxygen is low. HIF-1a increases fermentation capacity during hypoxia.

Governing equations:

- $o2_inhibition = 1 - O2 / (Km_o2_inh + O2)$
- $v_ferm = Vmax_ferm * ferm_scale * Pyr / (Km_pyr + Pyr) * o2_inhibition$
- $ferm_scale = 1 + (HIF_MAX_FERM - 1) * hif1a$

Implementation excerpt:

```
def compute(s, dt, v_oxphos):
    """
    v_oxphos is passed in so fermentation and OxPhos share pyruvate fairly.
    """
    ferm_scale = 1.0 + (HIF_MAX_FERM - 1.0) * s["hif1a"]
    o2_inhibition = 1.0 - mm(s["O2"], KM_FERM_O2_INH)

    v = VMAX_FERM * ferm_scale * mm(s["pyruvate"], KM_FERM_PYR) * o2_inhibition * dt
    # Pyruvate remaining after OxPhos
    v = min(v, max(s["pyruvate"] - v_oxphos, 0.0))
    return {"v_ferm": v}

def apply(s, fluxes):
    v = fluxes["v_ferm"]
    s["pyruvate"] -= v
    s["lactate"] += v
    return s
```

Process 5: ATP Demand

Represents housekeeping ATP use by processes not explicitly modeled.

Governing equations:

- $v_demand = ATP_DEMAND_BASE * ATP / (Km_demand + ATP)$
- $ATP \rightarrow ADP$

Implementation excerpt:

```
def compute(s, dt):
    v = ATP_DEMAND_BASE * mm(s["ATP"], KM_DEMAND_ATP) * dt
    v = min(v, s["ATP"])
    return {"v_demand": v}

def apply(s, fluxes):
    v = fluxes["v_demand"]
    s["ATP"] -= v
    s["ADP"] += v
    return s
```

Process 6: HIF-1a Oxygen Sensing

Models the oxygen-sensitive HIF-1a switch that couples hypoxia to glucose import and fermentation.

Governing equations:

- if $O_2 < HIF_O_2_THRESH$: $dHIF/dt = k_{rise} * (1 - HIF)$
- else: $dHIF/dt = -k_{fall} * HIF$
- HIF is clamped to [0, 1]

Implementation excerpt:

```
def compute(s, dt):
    if s["O2"] < HIF_O2_THRESH:
        delta = HIF_RISE_RATE * (1.0 - s["hif1a"]) * dt
    else:
        delta = -HIF_FALL_RATE * s["hif1a"] * dt
    new_hif = clamp(s["hif1a"] + delta, 0.0, 1.0)
    return {"hif1a": new_hif}

def apply(s, fluxes):
    s["hif1a"] = fluxes["hif1a"]
    return s
```

Process 7: Calcium Signalling

Maintains low resting cytosolic calcium using a leak plus pump homeostasis model.

Governing equations:

- $v_{leak} = CA_LEAK_RATE$
- $v_{pump} = CA_PUMP_RATE * \max(Ca_{cyt} - CA_REST, 0)$
- $dCa/dt = v_{leak} - v_{pump}$

Implementation excerpt:

```
def compute(s, dt):
    v_leak = CA_LEAK_RATE * dt
    v_pump = CA_PUMP_RATE * max(s["Ca_cyt"] - CA_REST, 0.0) * dt
    v_pump = min(v_pump, s["Ca_cyt"])
    return {"v_ca_leak": v_leak, "v_ca_pump": v_pump}

def apply(s, fluxes):
    s["Ca_cyt"] += fluxes["v_ca_leak"] - fluxes["v_ca_pump"]
    s["Ca_cyt"] = max(s["Ca_cyt"], 0.0)
    return s
```

Process 8: mTOR Protein Synthesis

Links ATP abundance and amino acids to protein synthesis through an mTOR-like anabolic gate.

Governing equations:

- $\text{mTOR_activity} = \text{clamp}(\text{ATP} / \text{NORMAL_ATP}, 0, 1)$
- $v_{\text{synth}} = V_{\text{max_ribo}} * \text{mTOR_activity} * \text{ATP} / (\text{Km_atp} + \text{ATP}) * \text{AA} / (\text{Km_aa} + \text{AA})$
- $4 \text{ ATP} + 1 \text{ amino acid} \rightarrow 1 \text{ protein_mass unit} + 4 \text{ ADP}$

Implementation excerpt:

```
def compute(s, dt):
    mtor = clamp(s["ATP"] / NORMAL_ATP, 0.0, 1.0)
    v_theory = VMAX_RIBO * mtor * mm(s["ATP"], KM_ATP_RIBO) * mm(s["amino_acids"], KM_AA) * dt
    v_actual = min(v_theory,
                   s["ATP"] / ATP_PER_AA,
                   s["amino_acids"])
    return {"v_synth": v_actual}

def apply(s, fluxes):
    v = fluxes["v_synth"]
    s["ATP"] -= ATP_PER_AA * v
    s["ADP"] += ATP_PER_AA * v
    s["amino_acids"] -= v
    s["protein_mass"] += v
    return s
```

Process 9: Na/K-ATPase Pump

Models Na/K-ATPase as a major ATP consumer that maintains sodium and potassium gradients.

Governing equations:

- $v_{\text{pump}} = V_{\text{max_pump}} * \text{pump_activity} * \text{mm}(\text{ATP}) * \text{Hill}(\text{Na_cyt}, \text{Km_Na}, 3) * \text{Hill}(\text{K_ext}, \text{Km_K}, 2)$
- $1 \text{ ATP} + 3 \text{ Na_cyt} + 2 \text{ K_ext} \rightarrow \text{ADP} + 3 \text{ Na_ext} + 2 \text{ K_cyt}$

Implementation excerpt:

```
def compute(s, dt, pump_activity=1.0):
    atp_drive = mm(s["ATP"], KM_ATP_PUMP)
    na_drive = hill(s["Na_cyt"], KM_NA, 3)
    k_drive = hill(s["K_ext"], KM_K, 2)

    v = VMAX_PUMP * pump_activity * atp_drive * na_drive * k_drive * dt
    v = min(v,
            s["ATP"],
            s["Na_cyt"] / 3.0,
            s["K_ext"] / 2.0)
    return {"v_pump": v}

def apply(s, fluxes):
    v = fluxes["v_pump"]
    s["ATP"] -= v
    s["ADP"] += v
    s["Na_cyt"] -= 3.0 * v
    s["Na_ext"] += 3.0 * v
    s["K_ext"] -= 2.0 * v
    s["K_cyt"] += 2.0 * v
    return s
```


Spatial Extension

The next architectural step is to replace one scalar concentration per molecule with a 3-D array per molecule. Each voxel evaluates the same local reaction equations and also exchanges material with its six face-neighbor voxels through a finite-difference Laplacian.

- $dC/dt = D * \text{Laplacian}(C) + R(C)$
- $\text{Laplacian}(C) = (C[x+1] + C[x-1] + C[y+1] + C[y-1] + C[z+1] + C[z-1] - 6 * C[x,y,z]) / dx^2$
- $dATP/dt = D_ATP * \text{Lap}(ATP) + v_oxphos - v_demand - v_synth - v_pump$
- $dO2/dt = D_O2 * \text{Lap}(O2) + v_import - 3 * v_oxphos$
- $dGlucose/dt = D_Glc * \text{Lap}(Glucose) + v_glut - v_glycolysis$
- $dLactate/dt = D_Lac * \text{Lap}(Lactate) + v_fermentation - v_export$

```
// One GPU invocation per voxel
pos = global_invocation_id
atp = readATP(pos)
lap_atp = (readATP(pos+X) + readATP(pos-X)
          + readATP(pos+Y) + readATP(pos-Y)
          + readATP(pos+Z) + readATP(pos-Z)
          - 6.0 * atp) / (dx * dx)

oxphos = compute_oxphos(pyr, o2, adp)
synth = compute_mTOR_synth(atp, amino_acids)
pump = compute_NaK_pump(atp, sodium, potassium_ext)

d_atp = D_ATP * lap_atp + oxphos - demand - synth - pump
writeATP_next(pos, atp + d_atp * sub_dt)
```

How To Continue

- Run simulate.py, validate.py, and plot.py after each biology or parameter change.
- Refine oxygen handling by making O2 exchange concentration-gradient driven, the same way GLUT is now reversible.
- Run normal, hypoxia, and starvation validations; tune only after preserving expected qualitative behavior.
- Add a 1-D oxygen diffusion prototype with the same Fick/Laplacian update before jumping to 3-D.
- Promote scalar state variables to small 32^3 NumPy arrays and graft the existing process functions over voxel arrays.
- Add membrane/boundary masks so transport only occurs at the cell surface while metabolism occurs inside the cell.
- Use explicit Euler sub-stepping with $dt < dx^2 / (2 * D_max)$, then move the stencil to CuPy/CUDA or WGSL.
- Connect WebGL/three.js to 3-D textures for ATP, O2, lactate, Na_cyt, and protein_mass.
- Make the final demo an interactive hypoxia scenario: lower external O2, show HIF-1a rise, lactate accumulation, ATP stress, pump failure, and recovery.

Deferred Scope

For a six-week MVP, keep calcium spikes, apoptosis/cell fate, transcription/translation detail, autophagy, Stokes flow, and Poisson electrostatics out of the required deliverable. Mention them as future work after the WebGL spatial hypoxia demo is stable.